

npm, TypeScript Types & Interfaces for Hotel Booking

Objectives

By the end of this session, you should be able to:

- Understand why TypeScript is crucial for hotel booking systems
 - Define TypeScript interfaces for hotel entities (Hotel, Room, Booking, Guest)
 - Use TypeScript type annotations for hotel data
 - Create enums for hotel constants (room types, booking status)
 - Implement type safety for hotel booking logic
 - Set up TypeScript configuration for hotel booking project
-

npm

What is npm?

npm = Node Package Manager

It is the default tool that comes with Node.js and is used to manage JavaScript/TypeScript libraries.

Simple explanation Think of npm as an app store for code: Instead of writing everything from scratch, you can install ready-made tools like:

- TypeScript
- React
- Express
- tsx
- ts-node

What npm does

1. Install packages

```
npm install typescript
```

2. Install globally (available anywhere)

```
npm install -g tsx
```

3. Run project scripts

```
npm run start
```

4. Manage dependencies It stores them in:

```
node_modules/  
package.json
```

Why it matters

Without npm, you would have to:

- download libraries manually
- manage versions yourself
- handle dependencies yourself npm automates all of that.

Why TypeScript for Hotel Booking?

Hotel booking systems deal with complex data structures:

- Multiple related entities (hotels, rooms, bookings, guests)
- Strict business rules (room capacity, pricing, availability)
- Financial calculations (total prices, discounts, taxes)
- Date handling (check-in/check-out, duration calculations)

TypeScript helps prevent costly bugs:

- **Type Safety:** Prevent assigning string to price field
 - **Interface Contracts:** Ensure all hotel data has required fields
 - **Compile-time Errors:** Catch issues before deployment
 - **Better IDE Support:** Autocomplete for hotel properties
 - **Refactoring Safety:** Change hotel structure with confidence
-

Setting Up TypeScript for Hotel Booking

1. Install TypeScript Compiler

```
npm install -g typescript
```

2. Create your TypeScript file Example:

```
hotel-booking.ts
```

Add your code inside this file.

3. Run TypeScript directly (Recommended)

Instead of compiling manually or using ts-node, use tsx (modern, fast, and Node 24 compatible). Install tsx:

```
npm install -g tsx
```

Run your file:

```
tsx hotel-booking.ts
```

 **Optional (Classic Compilation Method)** If you still want the traditional workflow:

Compile:

```
tsc hotel-booking.ts --outFile hotel-booking.js
```

Run:

```
node hotel-booking.js
```

Activity 1: TypeScript vs JavaScript for Hotel Booking

TypeScript vs JavaScript for Hotel Booking

JavaScript (Dynamic)

```
// Risky: No type checking
function calculateTotal(price, nights, discount) {
  return price * nights - discount; // What if discount is string?
}

const total = calculateTotal(200, 3, "10%"); // Runtime error!
```

TypeScript (Static)

```
// Safe: Type checking prevents errors
function calculateTotal(price: number, nights: number, discount: number):
number {
  return price * nights - discount;
}

const total = calculateTotal(200, 3, 10); // Works
// const total = calculateTotal(200, 3, "10%"); // Compile-time error!
```

Practice: Convert these JavaScript functions to TypeScript:

1. `calculateRoomPrice(price, nights)`

2. `validateGuest(guest)`
 3. `checkAvailability(room, date)`
-

Activity 2: Type Annotations for Hotel Data

Type Annotations for Hotel Data

Type annotations in TypeScript define the type of a variable or function value.

They help you:

- Catch errors early
- Improve code readability
- Make data structure clear and predictable

Common types: `string` | `number` | `boolean` | `string[]` | `{ [key: string]: string }`

Hotel Booking Type Examples:

```
// Basic type annotations
let hotelName: string = "Grand Palace Hotel";
let roomPrice: number = 200;
let availableRooms: number = 45;
let hasPool: boolean = true;
let amenities: string[] = ["WiFi", "Pool", "Spa"];

// Function with type annotations
function calculateTotalPrice(basePrice: number, nights: number): number {
    return basePrice * nights;
}

const total = calculateTotalPrice(200, 3); // 600
```

Example: return a list of room numbers

```
function getAvailableRooms(): number[] {
    return [101, 102, 103, 205];
}
```

```
const rooms = getAvailableRooms();  
console.log(rooms);
```

Example: return a list of hotel names

```
function getHotels(): string[] {  
    return ["Grand Palace", "City Inn", "Ocean View"];  
}
```

Function Returning an Object

Return a single hotel object:

```
function getHotel(): {  
    name: string;  
    rating: number;  
    location: string;  
} {  
    return {  
        name: "Grand Palace Hotel",  
        rating: 4.5,  
        location: "Foshan"  
    };  
}  
  
const hotel = getHotel();  
console.log(hotel);
```

Better Approach (Recommended)

Instead of repeating the object shape inline, define a **type**:

```
type Hotel = {  
    name: string;  
    rating: number;  
    location: string;  
};
```

Then use it:

```
function getHotel(): Hotel {  
    return {  
        name: "Grand Palace Hotel",
```

```
    rating: 4.5,  
    location: "Foshan"  
  };  
}
```

Practice: Create type annotations for:

- Guest information (name, email, phone)
- Room details (number, type, capacity, price)
- Booking dates (check-in, check-out)
- Hotel ratings and reviews

Activity 3: Interfaces for Hotel Entities

What are Interfaces?

Interfaces in TypeScript define the structure of an object.

They help you:

- Organize data models
- Enforce consistency
- Improve code readability
- Reduce bugs in large applications

Interfaces define the blueprint of real-world objects in your system.

Interfaces for Hotel Entities

Hotel Booking Interfaces:

```
// Hotel interface  
interface Hotel {  
  id: number;  
  name: string;  
  location: string;  
  rating: number;  
  price: number;  
  available: boolean;  
  amenities: string[];  
}
```

```

}

// Room interface
interface Room {
  roomNumber: string;
  type: RoomType;
  capacity: number;
  pricePerNight: number;
  available: boolean;
  amenities: string[];
}

// Guest interface
interface Guest {
  id: string;
  name: string;
  email: string;
  phone: string;
  preferences?: string[]; // Optional property
}

// Booking interface
interface Booking {
  bookingId: string;
  hotel: Hotel;
  room: Room;
  guest: Guest;
  checkIn: Date;
  checkOut: Date;
  totalPrice: number;
  status: BookingStatus;
}

```

```

type RoomType = "single" | "double" | "suite";

type BookingStatus = "pending" | "confirmed" | "cancelled";

```

Example Usage:

```

const booking: Booking = {
  bookingId: "B001",
  hotel: {
    id: 1,
    name: "Grand Palace",
    location: "Foshan",
    rating: 4.5,
    price: 200,
    available: true,
    amenities: ["WiFi", "Pool"]
  },
  room: {

```



```
    roomNumber: "101",
    type: "double",
    capacity: 2,
    pricePerNight: 200,
    available: true,
    amenities: ["AC", "TV"]
  },
  guest: {
    id: "G001",
    name: "Le",
    email: "le@example.com",
    phone: "+123456789",
    preferences: ["non-smoking"]
  },
  checkIn: new Date("2026-06-01"),
  checkOut: new Date("2026-06-03"),
  totalPrice: 600,
  status: "confirmed"
};
```

Note: In Activity 4 we replace `type RoomType = "single" | ...` with **enums** — better for a fixed list of hotel constants used across the app.

Practice: Create interfaces for:

- Payment information
- Hotel reviews
- Room availability calendar
- Guest loyalty program

Activity 4: Enums for Hotel Constants

What are Enums?

An **enum** defines a fixed set of named constants. In a hotel system, room types and booking statuses should only be valid values — not any random string.

```
// Risky – any string is accepted
let status: string = "confrimed"; // typo, no error until runtime

// Safe – only enum values allowed
let status: BookingStatus = BookingStatus.CONFIRMED;
```

Enum vs Union Type (from Activity 3)

Approach	Example	Best for
Union type	<code>type RoomType = "single" "double"</code>	Quick prototypes, 2–3 values
Enum	<code>enum RoomType { SINGLE = "single" }</code>	Hotel apps — shared constants, labels, API JSON

We use **string enums** so values match JSON from your API (`"confirmed"`, `"suite"`, etc.).

Hotel Booking Enums

```
enum RoomType {
  SINGLE = "single",
  DOUBLE = "double",
  SUITE = "suite",
  PENTHOUSE = "penthouse",
}

enum BookingStatus {
  PENDING = "pending",
  CONFIRMED = "confirmed",
  CHECKED_IN = "checked_in",
  CHECKED_OUT = "checked_out",
  CANCELLED = "cancelled",
}

enum PaymentMethod {
  CREDIT_CARD = "credit_card",
  DEBIT_CARD = "debit_card",
  PAYPAL = "paypal",
  BANK_TRANSFER = "bank_transfer",
}
```

Connect Enums to Interfaces

Update the `Room` and `Booking` interfaces from Activity 3:

```
interface Room {
  roomNumber: string;
```

```

type: RoomType;           // was: type RoomType string union
capacity: number;
pricePerNight: number;
available: boolean;
amenities: string[];
}

interface Booking {
  bookingId: string;
  hotel: Hotel;
  room: Room;
  guest: Guest;
  checkIn: Date;
  checkOut: Date;
  totalPrice: number;
  status: BookingStatus; // was: type BookingStatus string union
}

```

Using Enums in Functions

Display a human-readable room label:

```

function getRoomTypeLabel(type: RoomType): string {
  switch (type) {
    case RoomType.SINGLE:
      return "Single Room";
    case RoomType.DOUBLE:
      return "Double Room";
    case RoomType.SUITE:
      return "Suite";
    case RoomType.PENTHOUSE:
      return "Penthouse";
    default:
      return "Unknown";
  }
}

console.log(getRoomTypeLabel(RoomType.SUITE)); // "Suite"

```

Check if a booking can still be cancelled:

```

function canCancelBooking(status: BookingStatus): boolean {
  return (
    status === BookingStatus.PENDING ||
    status === BookingStatus.CONFIRMED
  );
}

```

```
console.log(canCancelBooking(BookingStatus.CHECKED_IN)); // false
```

Filter bookings by status:

```
const bookings: Booking[] = [/* your sample data */];

function getBookingsByStatus(
  bookings: Booking[],
  status: BookingStatus
): Booking[] {
  return bookings.filter((b) => b.status === status);
}

const confirmed = getBookingsByStatus(bookings, BookingStatus.CONFIRMED);
```

Iterate Over Enum Values

Useful for populating a room-type dropdown in the UI (Day 3):

```
const roomTypeOptions = Object.values(RoomType);
// ["single", "double", "suite", "penthouse"]
```

Follow-Along: Create **enums.ts**

```
# In your project folder from Activity 2
touch enums.ts
```

Add all three enums, then import them in your **types.ts**:

```
import { RoomType, BookingStatus, PaymentMethod } from "./enums";
```

Run and test:

```
tsx enums.ts
```

Practice (do these before Activity 5)

1. Add `enum AmenityCategory { ROOM = "room", HOTEL = "hotel", WELLNESS = "wellness" }`
 2. Write `getStatusLabel(status: BookingStatus): string` — returns "Pending", "Confirmed", etc.
 3. Write `isRoomAvailable(room: Room, bookingStatus: BookingStatus): boolean` — room must be `available: true` and status not `CANCELLED`
 4. Try assigning `RoomType.DOUBLE` to a variable typed as `RoomType` — then try `"double"` as a string and observe the TypeScript error
-

Activity 5: TypeScript Practice with Hotel Context

How to Use This Activity

1. **Try each question yourself first**
2. **Then** compare with the suggested answer
3. Run your file with `tsx hotel-practice.ts` to verify

Create `hotel-practice.ts` and work through all five questions in one file.

Question 1: Type a Price Calculator

JavaScript (unsafe):

```
function calculateRoomPrice(basePrice, nights, discount) {  
  return basePrice * nights - discount;  
}
```

Your task: Add types. `discount` is optional (default `0`). Return a `number`.

► Click to reveal answer

```
function calculateRoomPrice(  
  basePrice: number,  
  nights: number,  
  discount: number = 0  
): number {  
  return basePrice * nights - discount;  
}  
  
console.log(calculateRoomPrice(200, 3));           // 600  
console.log(calculateRoomPrice(200, 3, 50));       // 550  
// calculateRoomPrice(200, 3, "50");              // compile error
```

Question 2: Type a Greeting with Template Literals

JavaScript:

```
function greet(name) {  
  return "Hello, " + name + "!";  
}
```

Your task: Rename to `greetGuest(name, hotelName)` — both `string`, return a welcome message using a template literal.

► Click to reveal answer

```
function greetGuest(name: string, hotelName: string): string {  
  return `Welcome ${name} to ${hotelName}!`;  
}  
  
console.log(greetGuest("Le", "Grand Palace Hotel"));  
// "Welcome Le to Grand Palace Hotel!"
```

Question 3: Type an Array Reduction (Revenue)

JavaScript:

```
function sumArray(numbers) {  
  return numbers.reduce((acc, curr) => acc + curr, 0);  
}
```

Your task: Rename to `calculateTotalRevenue(roomPrices: number[])`. Use it with nightly prices from three booked rooms.

► Click to reveal answer

```
function calculateTotalRevenue(roomPrices: number[]): number {  
  return roomPrices.reduce((total, price) => total + price, 0);  
}  
  
const nightlyRates = [150, 200, 280];  
console.log(calculateTotalRevenue(nightlyRates)); // 630
```

Question 4: Interface + Optional Fields

JavaScript:

```
function printUser(user) {  
  console.log(user.name + " - " + user.email);  
}  
  
const guest = { name: "Le", email: "le@example.com" };
```

Your task:

- Define a `Guest` interface (`name`, `email`, optional `phone`, optional `loyaltyPoints`)
- Type `printGuestInfo(guest: Guest): void`
- Create two guests: one with phone, one without — both must type-check

► Click to reveal answer

```
interface Guest {  
  name: string;  
  email: string;  
  phone?: string;
```

```

    loyaltyPoints?: number;
}

function printGuestInfo(guest: Guest): void {
    const phone = guest.phone ?? "no phone";
    console.log(`${guest.name} - ${guest.email} (${phone})`);
}

const guestA: Guest = { name: "Le", email: "le@example.com", phone: "+86 138 0000 0001" };
const guestB: Guest = { name: "Chen", email: "chen@example.com" };

printGuestInfo(guestA);
printGuestInfo(guestB);

```

Question 5: Enums + Interfaces Together (Capstone)

Your task: Using `RoomType` and `BookingStatus` from Activity 4, write:

```

interface RoomSummary {
    roomNumber: string;
    type: RoomType;
    pricePerNight: number;
}

function createBookingSummary(
    room: RoomSummary,
    nights: number,
    status: BookingStatus
): string {
    // Return: "Room 201 (suite) - 3 nights - $840 - confirmed"
}

```

Hints:

- Use `RoomType` enum for `room.type`
- Multiply `pricePerNight * nights` for total
- Include `status` in the returned string

► Click to reveal answer

```

enum RoomType {
    SINGLE = "single",

```



```
DOUBLE = "double",
SUITE = "suite",
PENTHOUSE = "penthouse",
}

enum BookingStatus {
  PENDING = "pending",
  CONFIRMED = "confirmed",
  CANCELLED = "cancelled",
}

interface RoomSummary {
  roomNumber: string;
  type: RoomType;
  pricePerNight: number;
}

function createBookingSummary(
  room: RoomSummary,
  nights: number,
  status: BookingStatus
): string {
  const total = room.pricePerNight * nights;
  return `Room ${room.roomNumber} (${room.type}) - ${nights} nights -
  ${total} - ${status}`;
}

const room: RoomSummary = {
  roomNumber: "201",
  type: RoomType.SUITE,
  pricePerNight: 280,
};

console.log(createBookingSummary(room, 3, BookingStatus.CONFIRMED));
// Room 201 (suite) - 3 nights - $840 - confirmed
```

Self-Check Before Main Activity

- ☐ All five questions compile with `tsx hotel-practice.ts`
- ☐ You can explain when to use `interface` vs `enum`
- ☐ Wrong types (string instead of number, invalid status) show red squiggles in the IDE

Main Class Activity: Hotel Booking TypeScript System

Goal: Combine everything from today — **types**, **interfaces**, **enums**, and typed functions — into one runnable project.

You should already have from Activities 2–5:

- **hotel-practice.ts** (or similar) with typed functions
- Understanding of **Hotel**, **Room**, **Guest**, **Booking** interfaces
- **RoomType** and **BookingStatus** enums

Setup

```
mkdir hotel-booking-ts && cd hotel-booking-ts
npm init -y
npm install -D typescript tsx
npx tsc --init
```

Step 1: Create **enums.ts**

Move enums from Activity 4 here. Export **RoomType**, **BookingStatus**, **PaymentMethod**.

Step 2: Create **types.ts**

Define and export **Hotel**, **Room**, **Guest**, **Booking** — use enums from Step 1 for **room.type** and **booking.status**.

Step 3: Create **data.ts**

Add at least **2 hotels** and **3 rooms** as typed constants:

```
import type { Hotel } from "./types";

export const hotels: Hotel[] = [
  { id: 1, name: "Grand Palace Hotel", location: "Foshan", rating: 4.5,
    price: 200, available: true, amenities: ["WiFi", "Pool"] },
  // add more...
];
```

Step 4: Create `booking.ts`

Implement these typed functions:

Function	Purpose
<code>calculateBookingPrice(pricePerNight: number, nights: number): number</code>	Total cost
<code>validateBooking(booking: Booking): boolean</code>	Check dates, guest email, status
<code>generateBookingId(): string</code>	e.g. <code>BK-\${Date.now()}</code>
<code>getBookingsByStatus(bookings: Booking[], status: BookingStatus): Booking[]</code>	From Activity 4

Step 5: Create `main.ts` & test

```
import { hotels } from "../data";
import { calculateBookingPrice } from "../booking";

console.log("Hotels:", hotels.length);
console.log("3 nights at $200:", calculateBookingPrice(200, 3));
```

Run:

```
tsx main.ts
```

Step 6: Break TypeScript on purpose

In `main.ts`, try passing a string where a number is expected — confirm the IDE shows an error **before** you run the code.

Deliverable Checklist

- ☐ `enums.ts`, `types.ts`, `data.ts`, `booking.ts`, `main.ts` all compile

- ☐ `tsx main.ts` runs without errors
 - ☐ At least one function uses an **enum** parameter
 - ☐ At least one **interface** used for booking data
-

Preparation for Next Session

Read before next class:

- DOM Manipulation Guide: https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model
 - JavaScript DOM Tutorial: https://www.w3schools.com/js/js_htmlDOM.asp
-

Summary: TypeScript for Hotel Booking

Key Takeaways:

- **Type Safety:** Prevents costly bugs in hotel booking logic
- **Interfaces:** Define contracts for hotel data structures
- **Enums:** Standardize hotel constants (room types, booking status)
- **Compile-time Errors:** Catch issues before deployment
- **Better Developer Experience:** Autocomplete and refactoring support

Next Session: We'll use DOM manipulation to build interactive hotel booking interfaces.